UberClone — Versión Conductor

1. Descripción general

Desarrollé una aplicación móvil multiplataforma inspirada en Uber la versión de conductor, construida con React Native con expo para iOS y Android. El proyecto integra un ecosistema completo de APIs de Google para ofrecer una experiencia de movilidad fluida y en tiempo real.

El conductor recibe solicitudes de viaje en tiempo real, las atiende, cobra mediante una pasarela de pago y consulta sus ganancias netas después de la comisión de la plataforma.

Funcionalidades principales:

1. Registro del conductor con datos personales, del vehículo y documentos, con validaciones.
2. Cambio de estado Disponible / No disponible y envío de la ubicación en tiempo real.
3. Recepción de solicitudes con tiempo límite para aceptar o rechazar.
4. Navegación al punto de recogida y al destino con Google Directions API.
5. Cobro del viaje con Mercado Pago (recomendado) o Stripe en modo de prueba, y opción de pago en efectivo.
6. Cálculo automático de tarifa, comisión de la plataforma y ganancia neta del conductor.
7. Historial de carreras con filtros y resumen de ganancias por día, semana y mes.
8. Navegación combinada: Stack para el flujo del viaje, Tabs para el registro y Drawer para perfil, historial, ganancias y configuración.

2. Registro del conductor

El registro se divide en tres pestañas con navegación **Tab** (Datos personales, Vehículo, Documentos) y solo se habilita el botón **Enviar registro** cuando las tres están completas y válidas. Ningún campo puede quedar nulo o vacío; los errores se muestran debajo de cada campo, no en un `Alert`. Al enviar, la cuenta queda en estado **Pendiente de aprobación** y el conductor no puede ponerse Disponible hasta que el estado cambie a **Aprobado** (puede simularse cambiando el campo en la base de datos).

Pestaña 1 — Datos personales

Campo	Descripción y validación
Foto de perfil	Imagen desde cámara o galería; obligatoria
Nombre completo	Máximo 50 caracteres; solo letras y espacios
Documento de identidad	Numérico, entre 6 y 10 dígitos
Número de celular	Numérico, 10 dígitos
Fecha de nacimiento	Selector de fecha; el conductor debe ser mayor de 18 años
Género	Lista desplegable

Campo	Descripción y validación
Correo electrónico	Validación de @ y dominio
Idioma	Español / inglés (mismo selector de la app del pasajero)

Pestaña 2 — Vehículo

Campo	Descripción y validación
Categoría	Lista desplegable: Económico, XL, Premium (las mismas categorías que elige el pasajero)
Placa	Formato colombiano ABC123 para carro; convertir a mayúsculas antes de validar
Marca y modelo	Texto, máximo 30 caracteres cada uno
Año	Numérico, 4 dígitos, no mayor al año actual
Color	Lista desplegable
Capacidad de pasajeros	Numérico; XL exige mínimo 6
Foto del vehículo	Imagen obligatoria donde se vea la placa

Pestaña 3 — Documentos y pagos

Campo	Descripción y validación
Licencia de conducción	Número + foto; fecha de vencimiento posterior a hoy
SOAT	Foto + fecha de vencimiento posterior a hoy
Revisión técnico-mecánica	Foto + fecha de vencimiento (si aplica según el año del vehículo)
Correo de la cuenta de Mercado Pago	Correo donde el conductor recibiría sus pagos; validación de formato
Aceptación de términos	Checkbox obligatorio con el porcentaje de comisión visible

No se guardan números de cuenta bancaria ni de tarjeta en la app ni en AsyncStorage: los datos sensibles los maneja la pasarela de pago.

3. Pantallas del conductor

1. **Inicio:** mapa con la ubicación actual y un interruptor Disponible / No disponible. Mientras está Disponible, la ubicación se actualiza en la base de datos cada pocos segundos; al pasar a No disponible o salir de la pantalla, el seguimiento se detiene (función de limpieza de `useEffect`).
2. **Solicitud entrante:** tarjeta con origen, destino, distancia, duración, categoría, método de pago, tarifa estimada y **ganancia neta estimada** del conductor. Tiene un temporizador de 15 segundos; si llega a cero, la solicitud se rechaza automáticamente.
3. **Viaje en curso:** ruta hacia el punto de recogida; al tocar **Pasajero a bordo**, la ruta cambia hacia el destino. Botón para finalizar el viaje.
4. **Cobro y resumen del viaje:** muestra tarifa final, comisión, ganancia neta y estado del pago (Aprobado, Pendiente, Rechazado o Efectivo).
5. **Ganancias:** totales del día, la semana y el mes (ver sección 5).
6. **Historial de carreras:** listado con filtros (ver sección 6).
7. **Perfil y configuración:** datos editables, vehículo, documentos, idioma y cierre de sesión.

Estados de un viaje (se guardan en el campo `status` y controlan qué pantalla se muestra):

1. `requested` — el pasajero solicitó el viaje.
2. `accepted` — el conductor aceptó (si rechaza o se vence el tiempo, la solicitud pasa al siguiente conductor o queda `rejected`).
3. `driver_arriving` — el conductor va hacia el punto de recogida.
4. `in_progress` — el pasajero está a bordo.
5. `completed` — el viaje terminó y se calcularon tarifa, comisión y ganancia.
6. `paid` — el pago fue confirmado por la pasarela o registrado en efectivo.
7. `cancelled` — cancelado por cualquiera de las partes antes de `in_progress`.

Uso obligatorio de hooks vistos en clase:

- `useState` para el estado de disponibilidad, la solicitud activa y los formularios.
- `useEffect` con función de limpieza para el seguimiento de ubicación, el temporizador de la solicitud y las suscripciones en tiempo real.
- `useContext` o Redux para la sesión del conductor y el viaje activo.
- `useMemo` para los totales de ganancias y `useCallback` para los manejadores de aceptar y rechazar.
- Hooks personalizados mínimos: `useDriverLocation`, `useTripRequests` y `useEarnings`.
- AsyncStorage solo para preferencias y la sesión (idioma, último estado de disponibilidad), nunca para datos sensibles.

4. Pasarela de pagos

Se recomienda **Mercado Pago Checkout Pro** como pasarela principal, porque ofrece cuentas de prueba de vendedor y comprador con tarjetas de prueba (documentación de cuentas de prueba). **Stripe** se acepta solo en modo de prueba.

Criterio	Mercado Pago	Stripe
Entorno de pruebas	Cuentas de prueba + tarjetas de prueba	Modo de prueba con llaves `sk_test` / `pk_test`
Integración sugerida en React Native	Checkout Pro abierto en `react-native-webview` o el navegador	`@stripe/stripe-react-native` con PaymentSheet
Medios locales	Tarjetas, PSE, efectivo según el país	Principalmente tarjetas

Flujo de pago obligatorio (cualquiera de las dos pasarelas):

1. Al finalizar el viaje, la app calcula la tarifa final con la misma función usada en la estimación.
2. La app envía al **backend** (Node.js/Express o Firebase Cloud Functions) el `tripId` y el monto.
3. El backend crea la preferencia de pago (Mercado Pago) o el PaymentIntent (Stripe) usando la llave secreta.
4. La app abre el checkout y el pasajero paga con una tarjeta de prueba.
5. La pasarela notifica el resultado (URL de retorno o webhook) y el backend actualiza el viaje: `paymentStatus`, `paymentId` y `status: paid`.
6. La app del conductor escucha el cambio en tiempo real y muestra el resumen del cobro.

Reglas de seguridad:

- El `access_token` de Mercado Pago o la `secret key` de Stripe **nunca** van en el código de la app; solo en el backend, en un archivo `.env` excluido de Git.
- El repositorio incluye un `.env.example` con los nombres de las variables, sin valores.
- Solo se usan credenciales de prueba durante todo el proyecto.

Pago en efectivo: el conductor marca el cobro como recibido. Como la plataforma no recibió el dinero, la comisión se descuenta del saldo del conductor (puede quedar en negativo). El pago al conductor (liquidación) se **simula** con un saldo en la app; el pago dividido real entre plataforma y conductor (split payments) es un reto opcional de bonificación.

5. Cálculo de tarifa, comisión y ganancias

La plataforma cobra una **comisión del 20 %** sobre la tarifa del viaje; las propinas van completas al conductor. Todos los valores son configurables desde un único archivo (`src/config/pricing.js`) y se manejan en pesos colombianos enteros para evitar errores de decimales.

Parámetros de referencia (valores didácticos, no tarifas reales):

Parámetro	Valor
Tarifa base	3.500 COP
Valor por km	1.200 COP

Parámetro	Valor
Valor por minuto	250 COP
Tarifa mínima	6.000 COP
Multiplicador Económico / XL / Premium	1,0 / 1,4 / 1,8
Comisión de la plataforma	20 %

Fórmulas (distancia y duración salen de Google Distance Matrix API):

$$\text{Tarifa} = \max\left(\text{Mínima},\ (\text{Base} + \text{km} \cdot V_{km} + \text{min} \cdot V_{min}) \cdot M_{categoría}\right)$$

$$\text{Comisión} = \text{Tarifa} \cdot 0{,}20 \qquad \text{Ganancia neta} = \text{Tarifa} - \text{Comisión} + \text{Propina}$$

La tarifa se redondea al múltiplo de 100 COP más cercano antes de calcular la comisión.

Ejemplo: viaje de 8,5 km y 22 minutos, sin propina.

Concepto	Económico	XL
Base + distancia + tiempo	3.500 + 10.200 + 5.500 = 19.200	19.200
Multiplicador	× 1,0	× 1,4 = 26.880
Tarifa (redondeada)	19.200 COP	26.900 COP
Comisión 20 %	3.840 COP	5.380 COP
Ganancia neta del conductor	15.360 COP	21.520 COP

Requisitos de implementación:

- Una sola función pura `calculateFare({ distanceKm, durationMin, category, tip })` que devuelva `{ fare, commission, driverEarnings }`, usada tanto en la estimación del pasajero como en el cierre del viaje del conductor.
- Cada viaje guarda el `commissionRate` aplicado, para que un cambio futuro de porcentaje no altere el historial.
- La pantalla **Ganancias** muestra, para hoy, la semana y el mes: número de viajes, total facturado, total de comisiones, propinas, ganancia neta y promedio por viaje, calculados con `useMemo` sobre el historial.
- Se muestra el **saldo** del conductor: ganancias de viajes pagados por la pasarela menos comisiones de viajes en efectivo.

6. Historial de carreras

El historial se muestra con `FlatList`, ordenado del viaje más reciente al más antiguo, con filtros **Hoy, Semana, Mes y Rango de fechas** y un filtro por método de pago. Cada elemento muestra fecha y hora, origen → destino, categoría, tarifa, ganancia neta y estado del pago; al tocarlo se abre el detalle con la ruta dibujada en el mapa, duración, distancia, comisión, propina y calificación del pasajero.

Modelo de datos sugerido (colección `trips` en Firestore o MongoDB; nombres en inglés):

Campo	Tipo	Descripción
`driverId`, `passengerId`	string	Referencias a los usuarios
`origin`, `destination`	object	`{ lat, lng, address }`
`distanceKm`, `durationMin`	number	Valores de Distance Matrix
`category`	string	`economy`, `xl`, `premium`
`fare`, `commission`, `tip`, `driverEarnings`	number	Pesos enteros
`commissionRate`	number	Porcentaje aplicado en ese viaje (0.20)
`paymentMethod`	string	`mercadopago`, `stripe`, `cash`
`paymentStatus`, `paymentId`	string	Respuesta de la pasarela
`status`	string	Estado del viaje (sección 3)
`rating`	number	Calificación de 1 a 5
`createdAt`, `completedAt`	timestamp	Fechas del viaje

Si el conductor no tiene viajes en el filtro elegido, se muestra un mensaje vacío en lugar de una lista en blanco. La lista debe paginar o limitar la consulta (por ejemplo, 20 viajes por carga).

7. APIs gratuitas adicionales

Sí conviene sumar tres servicios gratuitos obligatorios (notificaciones, tiempo real y geolocalización) y dejar los demás como opcionales. Desde marzo de 2025 Google Maps Platform ya no da un crédito mensual de 200 USD; cada API tiene su propio cupo gratuito, en general 10.000 eventos al mes en la categoría Essentials, mientras que los SDK de mapas para Android e iOS siguen siendo de uso ilimitado (FAQ de Google, desglose de precios). Para un proyecto de clase ese cupo alcanza, siempre que no se llame Directions o Distance Matrix en cada actualización de ubicación.

API o servicio	Uso en la versión conductor	Costo	Decisión
Firebase Cloud Messaging (`@react-native-firebase/messaging`)	Notificar nuevas solicitudes con la app en segundo plano	Gratis	Obligatoria
Firestore en tiempo real (`onSnapshot`) o MongoDB Change Streams	Sincronizar ubicación del conductor y estado del viaje	Plan gratuito con cuotas	Obligatoria
Geolocalización del dispositivo (`@react-native-community/geolocation`, `watchPosition`)	Ubicación continua del conductor	Gratis	Obligatoria
Maps SDK for Android / iOS (`react-native-maps`)	Mapa del conductor	Ilimitado	Ya incluida
Cloudinary (plan gratuito)	Fotos de perfil, vehículo y documentos	Gratis con límites	Recomendada
Nominatim (OpenStreetMap)	Convertir coordenadas en dirección	Gratis, máximo 1 petición por segundo	Opcional
OSRM u OpenRouteService	Ruta alternativa para ahorrar cupo de Directions	Gratis con límites	Opcional

Recomendaciones de consumo:

- Llamar Distance Matrix una sola vez al crear la solicitud y una vez al aceptarla; no en cada movimiento.
- Restringir la API key de Google por aplicación y activar alertas de presupuesto en Google Cloud.
- No usar Static Maps para cada elemento del historial: dibujar la ruta guardada con `Polyline` en `react-native-maps`.
- Firebase Storage puede exigir el plan de pago Blaze en proyectos nuevos; por eso se sugiere Cloudinary para las imágenes.

8. Instructivo de la bitácora de IA

Cada equipo debe documentar todo uso de herramientas de inteligencia artificial (Claude, ChatGPT, Copilot, Gemini u otras) en una bitácora que se entrega con el proyecto. Usar IA está permitido; no

declararlo se considera falta de honestidad académica. Si el equipo no usó IA, la bitácora igual se entrega con esa declaración.

Dónde se incluye:

- Archivo `docs/AI_LOG.md` en el repositorio, dentro de la rama `main`.
- Cada integrante registra sus entradas en su propia rama y las sube a `main` mediante Pull Request, igual que el código.
- El `README.md` tiene una sección **Uso de IA** con el enlace a la bitácora.
- En la entrega de la plataforma se adjunta la bitácora exportada a PDF.

Cómo generarla:

1. Registrar la entrada en el momento de usar la herramienta, no al final del proyecto.
2. Copiar el prompt tal como se escribió, sin resumirlo.
3. Resumir la respuesta en 2 o 3 líneas y, si la herramienta lo permite, pegar el enlace a la conversación.
4. Indicar qué se usó tal cual, qué se modificó y qué se descartó.
5. Anotar los archivos afectados y el hash del commit donde quedó el cambio.
6. Describir cómo se verificó que funcionaba (prueba en emulador, caso de prueba, revisión de documentación).
7. Marcar en el código los bloques asistidos con un comentario en inglés que remita a la entrada: `// AI-assisted: see docs/AI_LOG.md #7`.

Plantilla de cada entrada:

```
### Entrada #7
- Fecha: 2026-10-20
- Integrante: Nombre Apellido
- Herramienta: Claude
- Objetivo: Crear la función calculateFare con comisión configurable
- Prompt: "..." (texto completo)
- Resumen de la respuesta: ...
- Qué se usó / modificó / descartó: ...
- Archivos y commit: src/utils/calculateFare.js — a1b2c3d
- Verificación: Probado con el ejemplo de 8,5 km y 22 min (sección 5)
- Aprendizaje: Qué entendí y qué haría distinto
```

Reglas:

- Nunca pegar en un prompt llaves de API, `access_token` ni datos personales reales.
- En la sustentación, cualquier integrante puede ser llamado a explicar un bloque marcado como asistido por IA.

9. Entregables y rúbrica

Entregables:

- Enlace al repositorio con ramas por integrante, Pull Requests revisados y merge en `main`.
- `README.md` con instrucciones de instalación, variables de entorno (`.env.example`) y sección Uso de IA.

- `docs/AI_LOG.md` y su versión en PDF.
- Video de máximo 5 minutos con el flujo completo: registro del conductor, solicitud, viaje, pago de prueba e historial.

Fuentes

- Mercado Pago Colombia — Cuentas de prueba de Checkout Pro
- Platzi — Disponibilidad de Stripe en Colombia
- Google Maps Platform — Preguntas frecuentes de facturación
- StoreRocket — Precios de Google Maps API